

USE CASE DOCUMENT

Login Application

Production-Style Single-JAR Auth Service
(Spring Boot 3 · React 18 · Supabase Postgres)

Document	USECASE — Login Application
Version	1.0.0
Owner	AIGenXLab
Repository	github.com/aigenxlab-GH/Login-app
Local URL	http://localhost:8085

1. Executive Summary

The **Login Application** is a production-grade, role-based user authentication and management service. It is shipped as a single executable Spring Boot JAR that bundles both the REST API and the React SPA, and uses Supabase Postgres for persistence. The application supports Admin and General user roles, server-side session management, Argon2id password hashing with a server-side pepper, account lockout after repeated failed login attempts, and a complete administrator dashboard for managing user activation, employee IDs, and account removal.

1.1 Goals

- Provide a secure email + password sign-in flow with proper account lifecycle (sign up → admin activation → login → optional change password → logout).
- Allow administrators to manage every user account through a single enterprise-style dashboard with filter ribbon and pagination.
- Ship the entire application as one deployable artifact (a fat JAR) that requires only Java to run — no separate frontend host.
- Keep the database schema reproducible across environments via Flyway SQL migrations.

1.2 Non-Goals

- OAuth / SSO / social login — out of scope for this version.
- Multi-tenant isolation — single-tenant only.
- Email-based password reset — current change-password flow requires the old password.

2. Actors & Roles

Actor	Role	Capabilities
Anonymous Visitor	—	Sign Up, Sign In, Change Password (if old password is known)
General User	GENERAL	View own profile (Employee ID, name, email, designation, address, role) on the Home page; logout.
Administrator	ADMIN	All General-user capabilities, plus: list all other users, activate / deactivate user accounts (auto-assigns employee IDs), remove user accounts, view individual user detail pages.

3. High-Level Architecture

A single Spring Boot fat JAR exposes the REST API at `/api/**` and serves the compiled React SPA from the same origin. The browser therefore never needs CORS or a separate frontend host.

```
login-app.jar (single deployable)
├── Spring Boot 3.3.x · Java 21
├── /api/auth/**    login · signup · me · logout · change-pw
├── /api/admin/**  list/get/update/delete users (ADMIN only)
└── /**           static handler → React SPA
                  SpaFallbackController forwards SPA routes
                  (/home, /signup, /change-password,
                  /user/:id) to /index.html

Persistence: Supabase Postgres via JDBC
Schema: Flyway migrations V1 → V8
Sessions: app_sessions table + SESSION httpOnly cookie
Passwords: Argon2id (3/64MB/1) + server-side pepper
```

3.1 Technology Stack

Layer	Technology
Backend runtime	Spring Boot 3.3.x · Java 21 · Maven
Authentication	Spring Security 6 · Argon2id + pepper · custom DB session
Persistence	Spring Data JPA + raw JDBC (NamedParameterJdbcTemplate)
Database	Supabase Postgres (JDBC, never Supabase Auth)
Migrations	Flyway (SQL files in <code>db/migration/</code>)
Frontend	React 18 · Vite 5 · TypeScript (strict) · Tailwind 3
Routing	react-router-dom v6
Package manager	npm (lock file required)

4. Application Pages & Routes

Route	Access	Purpose
/ (LoginPage)	Public	Sign-in form (email + password) with links to Sign Up, Change Password, the use-case PDF download and the GitHub repository.
/signup (SignupPage)	Public	Create a new account: name, email, password, confirm password, address, designation, role (Admin/General).
/change-password	Public	Change password using email + old password + new password + confirm. Revokes all existing sessions for that user.
/home (HomePage)	Protected	General: own profile card. Admin: enterprise user-management table with filter ribbon, pagination and per-row Activate/Deactivate/Remove actions.
/user/:id (UserDetailPage)	Admin only	Single-user detail view (all fields except password) with inline status toggle and Remove button.

5. Use-Case Catalog

UC-01 — Sign Up

Actor: Anonymous visitor.

Pre-condition: The user does not already have an account.

Trigger: User clicks *Sign Up* on the login page.

Main flow:

1. User fills in name, email, password, confirm password, address, designation and selects a role (Admin / General).
2. Client-side validation runs on blur and on submit (required, email format, length limits, password match).
3. Server validates again with Jakarta annotations and persists the user with `is_active = false`.
4. If this is the very first user ever, the account is auto-activated and assigned **Employee ID 5001**.
5. User is redirected to the login page with a success banner.

Failure paths: `EMAIL_ALREADY_EXISTS` (409), `VALIDATION_FAILED` (400 with per-field `details`).

UC-02 — Sign In

Pre-condition: Account exists and is active.

Main flow:

1. User enters email + password and submits.

2. Server fetches the `app_users` row by lower-cased email.
3. If `is_active = false` → return `ACCOUNT_NOT_ACTIVATED` (403).
4. If `locked_until > now` → return `ACCOUNT_LOCKED` (403).
5. Argon2id verifies the supplied password (with pepper) against `password_hash`. On mismatch the `failed_login_attempts` counter increments; on the 5th failure `locked_until` is set 15 minutes in the future.
6. On success: failed-attempt counter is reset, a row in `app_sessions` is created with a UUID token, and the token is returned in the `SESSION` httpOnly cookie.
7. User is redirected to `/home`.

UC-03 — View Home Page (General)

A General user sees a single dashboard card with their own Employee ID, name, email, designation, address, role, and a Logout button. No other users are visible.

UC-04 — Manage Users (Admin)

An Admin's home page shows a wide table of every other user with the following enterprise controls:

- **Filter ribbon:** criterion dropdown (*By Name / By ID / By Status*) with a dynamic input beside it (text for Name/ID, status dropdown for Status). Filter only fires on the explicit *Apply Filter* button click (or pressing Enter in the text input). A *Reset All Filters* button clears everything.
- **Pagination footer:** *Rows per page* dropdown (5/10/15), live *X–Y of Z* counter, up to five page-number buttons in a sliding window, and *< Prev / Next >* controls.
- **Columns:** row number, Employee ID (highlighted chip), Name (clickable → user-detail page), Email, Designation, Status badge with inline Activate/Deactivate button, and Action column with a Remove button (confirmation modal).

UC-05 — Activate / Deactivate User (Admin)

1. Admin clicks the Activate button beside an inactive user.
2. Server calls `PATCH /api/admin/users/{id}/status` with `{ active: true }`.
3. If the user has no `employee_id` yet, one is allocated: `COALESCE(MAX(employee_id), 5000) + 1` in the range 5001–5999. Once assigned, the ID is permanent — even if the user is later deactivated.
4. If the admin deactivates a user, all that user's active sessions are revoked immediately so they're forced to log out.

Constraint: an Admin cannot change their own activation status or delete their own account (server enforces this with a 400 `BAD_REQUEST`).

UC-06 — View User Detail (Admin)

Clicking a user's name in the admin table opens `/user/{id}` with all stored fields (Employee ID, email, designation, address) plus the Status toggle and Remove button. A *Back to List* button returns to the dashboard.

UC-07 — Remove User (Admin)

1. Admin clicks Remove → confirmation modal appears.
2. On confirm, server revokes all of that user's sessions, then deletes the `app_users` row (cascade-deletes any remaining session rows).

UC-08 — Change Password

Anyone who knows their current password can change it via `/change-password`. The new password is hashed with Argon2id (with pepper) and all existing sessions for that user are revoked, forcing every other browser to re-authenticate.

UC-09 — Logout

`POST /api/auth/logout` revokes the current session row and returns a Set-Cookie that expires the `SESSION` cookie. The user is redirected to the login page with a "You logged out" banner.

6. REST API Reference

6.1 Auth endpoints (/api/auth/**)

Method	Path	Auth	Description
POST	/api/auth/login	Public	Authenticates, sets the SESSION cookie, returns the user.
POST	/api/auth/signup	Public	Creates an inactive account (first user is auto-activated and gets Employee ID 5001).
GET	/api/auth/me	Session	Returns the currently authenticated user.
POST	/api/auth/logout	Session	Revokes the session and clears the cookie.
POST	/api/auth/change-password	Public	Changes the password and revokes all existing sessions for that user.

6.2 Admin endpoints (/api/admin/** — ADMIN role)

Method	Path	Description
GET	/api/admin/users	List all users.
GET	/api/admin/users/{id}	Fetch a single user by UUID.
PATCH	/api/admin/users/{id}/status	Activate or deactivate a user (auto-assigns Employee ID on first activation).
DELETE	/api/admin/users/{id}	Delete the user and revoke all their sessions.

6.3 Standard error envelope

```
{
  "code": "EMAIL_ALREADY_EXISTS",
  "message": "An account with this email already exists.",
  "details": { "email": "..." } // present only for VALIDATION_FAILED
}
```

6.4 Catalogued error codes

Code	HTTP	Meaning
VALIDATION_FAILED	400	One or more DTO fields failed Jakarta validation; <code>details</code> map carries the field errors.
BAD_REQUEST	400	A general business-rule violation.
INVALID_CREDENTIALS	401	Email not found, or wrong password.
SESSION_TIMED_OUT	401	Session cookie missing, expired, idle > 480 min, or revoked.
FORBIDDEN	403	Caller lacks the required role for an endpoint.
ACCOUNT_LOCKED	403	Locked after 5 failed attempts (15-minute lockout).
ACCOUNT_NOT_ACTIVATED	403	Account exists but <code>is_active = false</code> ; ask Admin to activate.
EMAIL_ALREADY_EXISTS	409	Sign-up attempted with an email that already has an account.
OLD_PASSWORD_INVALID	422	Change-password failed because the supplied current password is wrong.
PASSWORD_MISMATCH	422	New password and confirmation do not match.
EMPLOYEE_ID_EXHAUSTED	422	All 999 IDs in 5001–5999 have been assigned (theoretical at this scale).
INTERNAL_ERROR	500	Unexpected exception (logged on the server, never leaks a stack trace).

7. Database Schema

7.1 app_users

Column	Type	Notes
id	UUID	Primary key, default <code>gen_random_uuid()</code> .
employee_id	INTEGER	Nullable, unique, range 5001–5999. Auto-assigned on first activation (or at signup for the first user).
name	TEXT	Required, 2–120 chars.
email	TEXT	Unique, lower-cased on insert.
password_hash	TEXT	Argon2id encoded string.
address	TEXT	Required, max 500 chars.
designation	TEXT	Required, 2–120 chars.
role	TEXT	<code>ADMIN</code> or <code>GENERAL</code> .
is_active	BOOLEAN	Default <code>false</code> .
failed_login_attempts	INT	Reset to 0 on success.
locked_until	TIMESTAMPTZ	Null when not locked.
password_updated_at	TIMESTAMPTZ	Set when the password is changed.
created_at / updated_at	TIMESTAMPTZ	Auditing timestamps.

7.2 app_sessions

Column	Type	Notes
id	UUID	Primary key.
user_id	UUID	FK → app_users.id , ON DELETE CASCADE.
session_token	TEXT	Unique. Stored verbatim in the SESSION cookie.
created_at	TIMESTAMPTZ	Session creation.
last_seen_at	TIMESTAMPTZ	Updated on every authenticated request — drives the 480-minute idle timeout.
revoked_at	TIMESTAMPTZ	Null = active. Set by logout, password change, deactivation, or deletion.
ip_address / user_agent	TEXT	Captured on session creation for audit.

7.3 Flyway migration history

Version	Description
V1	Initial app_user table.
V2	Rename app_user → app_users .
V3	Add app_sessions table.
V4	Add lockout columns + role (ADMIN/GENERAL).
V5	Truncate all data + add is_active .
V6	Drop and recreate both tables with employee_id .
V7	Truncate all data — fresh start for first-user employee-ID flow.
V8	Wipe all data again — clean slate after a stale admin account.

8. Security Model

8.1 Password storage — Argon2id with pepper

Passwords are hashed with **Argon2id** (3 iterations, 64 MB memory, parallelism 1 — OWASP minimum recommendation) via the `de.mkammerer:argon2-jvm` library. Before hashing, a server-side **pepper** from the `AUTH_PASSWORD_PEPPER` environment variable is concatenated to the raw password. A leaked database alone is therefore useless to an offline attacker who lacks the pepper.

Operational rule: the pepper must remain stable for the life of the deployment. Changing it (or restarting the JAR without it) invalidates every stored password hash. Always pass `--auth.password.pepper=<value>` when starting the JAR.

8.2 Sessions

- Stored in the `app_sessions` table, not in JWTs.
- Cookie name: `SESSION`, `HttpOnly`, `SameSite=Lax`. `Secure` is enabled automatically when the active Spring profile is anything other than `local`.
- Idle timeout: **480 minutes** (8 hours). `SessionAuthFilter` updates `last_seen_at` on every authenticated request.
- Stale cookies on API requests return `401 SESSION_TIMED_OUT`; on page requests the cookie is silently expired and the request continues so the login page loads normally.

8.3 Account lockout

After `auth.lockout.max-failed-attempts` (default 5) failed sign-in attempts, the account is locked for `auth.lockout.lockout-minutes` (default 15). A successful login resets the counter and clears the lock.

8.4 CSRF

CSRF protection is disabled because every endpoint is JSON-only and the `SESSION` cookie uses `SameSite=Lax`. If a cross-origin client is ever added, CSRF must be re-enabled with a double-submit token strategy.

8.5 Input validation

- Server: every request DTO carries Jakarta annotations (`@NotBlank`, `@Email`, `@Size`, `@Pattern`) with user-friendly messages.
- Client: `frontend/src/lib/validation.ts` mirrors those constraints. Errors only display after a field is touched (`onBlur`) or after submit; they live-clear as input becomes valid. Server `VALIDATION_FAILED` responses with a `details` map populate per-field errors automatically.

9. Build & Deployment

9.1 Single-JAR build pipeline

1. `maven-clean-plugin` wipes `backend/src/main/resources/static/` .
2. `frontend-maven-plugin` runs `npm install` then `npm run build` .
3. Vite is configured to write its output **directly** to `backend/src/main/resources/static/` (`outDir: '../backend/src/main/resources/static'`).
4. Maven's default `process-resources` copies `src/main/resources/**` (including the new `static/`) to `target/classes/` .
5. `maven-antrun-plugin` at `prepare-package` phase re-copies `static/` to `target/classes/static/` and verifies `index.html` is present (fail-fast guard against the Windows file-flush race condition).
6. `spring-boot:repackage` produces `backend/target/login-app.jar` with everything bundled under `BOOT-INF/classes/` .

9.2 Required environment variables

Variable	Required	Description
<code>DATABASE_URL</code>	yes	JDBC URL — e.g. <code>jdbc:postgresql://<host>:6543/postgres?sslmode=require</code>
<code>DATABASE_USERNAME</code>	yes	Postgres username — e.g. <code>postgres.<ref></code>
<code>DATABASE_PASSWORD</code>	yes	Postgres password.
<code>AUTH_PASSWORD_PEPPER</code>	yes	Long random hex string. Generate once with <code>openssl rand -hex 32</code> and never change it.
<code>AUTH_MAX_FAILED_ATTEMPTS</code>	no	Default 5.
<code>AUTH_LOCKOUT_MINUTES</code>	no	Default 15.
<code>PORT</code>	no	Default 8085.
<code>COOKIE_SECURE</code>	no	<code>true</code> in HTTPS prod; <code>false</code> locally.

9.3 Local prod-mode run

```
cd backend
.\mvnw.cmd clean package

java -jar target\login-app.jar `
  "--spring.datasource.url=jdbc:postgresql://<host>:6543/postgres?sslmode=require" `
  "--spring.datasource.username=postgres.<ref>" `
  "--spring.datasource.password=<password>" `
  "--auth.password.pepper=<long-random-string>"

# Open http://localhost:8085
```

10. First-Time Bootstrap

1. Open `http://localhost:8085`.
2. Click *Sign Up* and create the first account — choose **Admin** role.
3. That first user is automatically activated and is automatically assigned **Employee ID 5001**.
4. Log in and use the Admin dashboard to manage subsequent users.
5. All other users who sign up after the first one start as *inactive* (no employee ID). Their employee ID is assigned sequentially (5002, 5003, ...) the first time the Admin activates them.

11. References

- **Source repository:** <https://github.com/aigenxlab-GH/Login-app.git>
- **Conventions:** `CLAUDE.md` (project root)
- **Architecture decisions:** `docs/architecture.md`
- **Local run guide:** `docs/LOCAL_RUN.md`
- **Project README:** `README.md`